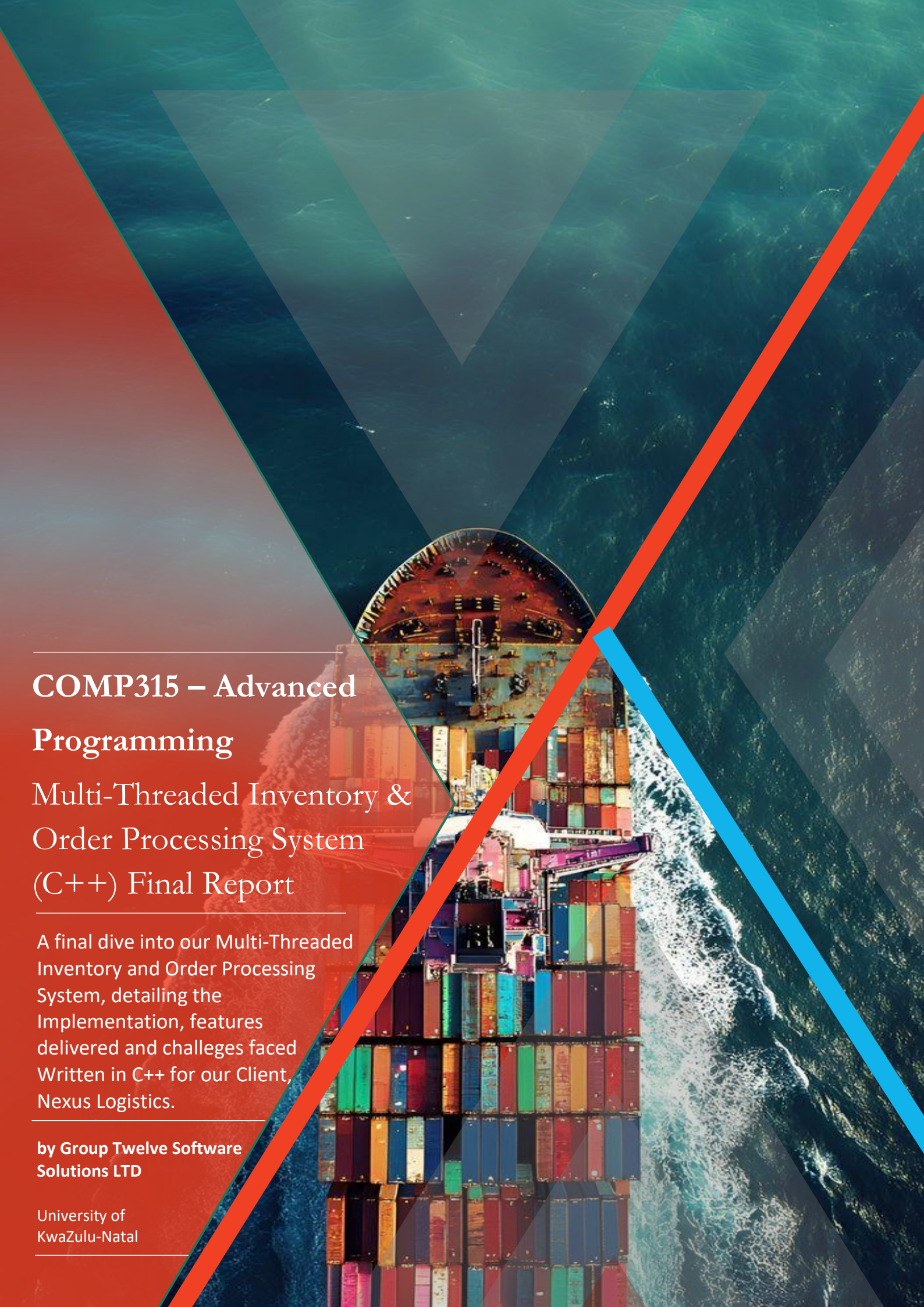




COMP315 – Advanced Programming

Multi-Threaded Inventory & Order Processing System (C++) Final Report

A final dive into our Multi-Threaded Inventory and Order Processing System, detailing the Implementation, features delivered and challenges faced
Written in C++ for our Client, Nexus Logistics.

by **Group Twelve Software
Solutions LTD**

University of
KwaZulu-Natal

Project Formalities

Multi-Threaded Inventory and Order Processing System (C++)

Group 12

Group Members & Contribution Percentage (Final):

Member Name	Student ID	Role	% Contribution
Yahya Bobat	223020664	Lead Developer	15
Jawad Kachbal	223037045	High-Level System Design Flow & Developer	15
Yusuf Sheikh	224068106	Documentation, Presentation	14
Mahomed Valli Mahomed	224005878	Overall Class & System Design and Diagram Design	14
Muhammad Varachia	224004509	Overall Class & System Design and Presentation	14
Kiyan Krishna	224115592	Overall Assistant in various Categories and System Testing	14
Andiswa Hlengwa	221002430	GUI Design and Implementation	14

Table of Contents

Project Formalities	1
Table of Figures.....	2
Introduction	3
Task Definition	3
.....	5
System Architecture.....	6
MULTI-THREADED REAL-TIME INVENTORY & TRADING SYSTEM PROCESS OVERVIEW	7
Class Design & OOP Structure.....	10
Overview	11
Justification for Virtual Functions	14
Conclusion.....	14
Data Structures & STL Usage	15
Performance	17
Concurrency Design & Thread Safety	18
Memory Management Strategy.....	21
User Interface Design.....	23
System Functionality & Testing.....	24
Performance & Limitations	27
Performance	27

Challenges & Solutions.....	28
Debugging Multi-Threaded Code.....	28
Race Conditions and Deadlock Prevention	28
STL Misuse and Refactoring.....	28
Design Changes During Implementation	29
Individual Contribution Summary.....	30
Conclusion.....	31
Final System Overview	31
AI Usage	32
References	33

Table of Figures

FIGURE 1: A FLOWCHART DETAILING THE FLOW OF DATA THROUGH THE SYSTEM AS WELL AS HOW COMPONENTS INTERACT WITH EACH OTHER.....	6
FIGURE 2 A DIAGRAM DETAILING THE STARTUP OF THE PROGRAM	7
FIGURE 3 AN EXAMPLE OF THE STRUCTURE OF THE PRODUCTS.CSV TAKEN FROM EXCEL.....	7
FIGURE 4 THE ORDERPOOL USING AN STL QUEUEZ	7
FIGURE 5 AN EXAMPLE OF THE ORDER STRUCTURE OF THE WAREHOUSEN_ORDERS.CSV TAKEN FROM EXCEL. THE DISPLAYED ENTRY CONTAINS 2 PRODUCTS IN THE ORDER CART WITH THE TRAILING QUANTITY REQUESTED – PRODUCTID,QTY,PRODUCTID,QTY 7	
FIGURE 6 OVERVIEW OF THE THREAD PIPELINE.....	8
FIGURE 7 A UML CLASS DIAGRAM OVERVIEW SHOWCASING VARIOUS CLASSES	10
FIGURE 8 THE PRODUCT BASE CLASS.....	12
FIGURE 9 DERIVED CLASSES FROM THE BASE PRODUCT CLASS	12
FIGURE 10 THE INVENTORY SYSTEM CLASS.....	12
FIGURE 11 THE ORDER CLASS.....	13
FIGURE 12 THE ORDERPOOL CLASS	13
FIGURE 13 THE ANALYTICSRECORD CLASS.....	13
FIGURE 14 THE ANALYTICSMANAGER CLASS.....	13
FIGURE 15 THE MAIN	14
FIGURE 16 AN EXTRACT FROM OUR INVENTORYSYSTEM HEADER FILE SHOWING HOW THE PRODUCTS DATA STRUCTURE IS USED TOGETHER WITH SHARED_PTRS AND MUTEX.....	15
FIGURE 17 AN EXTRACT OF A QUEUE FROM OUR ORDERPOOL HEADER	16
FIGURE 18 AN EXTRACT FROM OUR ORDER CLASS HEADER	16
FIGURE 19 AN EXTRACT FROM OUR INVENTORYSYSTEM HEADER	16
FIGURE 20 AN EXTRACT FROM OUR INVENTORYSYSTEM THAT USES FIND_IF.....	17
FIGURE 21 AN EXTRACT FROM OUR INVENTORYSYSTEM SHOWING HOW ONE OF THE SORTING FUNCTIONS USE STD::SORT.....	17
FIGURE 22 AN EXTRACT FROM THE ORDERPOOL CLASS.....	18
FIGURE 23 AN EXTRACT FROM THE INVENTORYSYSTEM CLASS.....	19
FIGURE 24 AN EXTRACT FROM THE WORKERTHREAD WHERE STATUSES ARE GIVEN.....	20
FIGURE 25 AN EXTRACT OF THE CONSOLE SHOWING THE ORDERS BEING PROCESSED	20
FIGURE 26 THE SYSTEMS MAIN MENU.....	23
FIGURE 27 A LOG DETAILING THE WORKER ID, THREAD ID, ORDER DETAILS AND ORDER STATUS.....	23
FIGURE 28 AN EXAMPLE OF SYSTEM FLOW WHEN THE USER SELECTS OPTION 1.....	23
FIGURE 29 AN EXTRACT SHOWING THE STOCK LEVELS FOR ITEM 1 AND 2 AFTER ORDER PROCESSING.....	23
FIGURE 30 AN EXTRACT SHOWING THE STOCK LEVELS FOR 1 AND 2 BEFORE ORDER PROCESSING.....	23
FIGURE 31 A TABLE OUTLINING TESTS CONDUCTED ON THE SYSTEM.....	25

Introduction

Task Definition

Group Twelve Software Solutions LTD has been contracted by Nexus Logistics to build a High-performance Multi-Threaded Inventory and Order Processing System. The system is required to handle concurrent order processing from multiple different warehouses. We have created a centralized Inventory and Order Processing System, uniting all warehouses, solving the problem of the inconsistency created by running multiple different warehouse Management Systems, allowing for faster order processing, and consistency when passing order information to our Client's partners further down the pipeline. The centralised system provides our Client with the ease of Inventory Management with concurrent order Processing using Multi-Threading and a compilation of order analytics. The use of Multi-Threading was used to achieve simultaneous order processing across connected Warehouses.

Primary System Objectives

- ✚ Providing a Centralised Inventory across all warehouses
- ✚ Allow for a consistent order format to be implemented for all warehouses to follow
- ✚ Allowing the user to keep track of stock and order transactions across their warehouses through a single Avenue, with the removal and addition of stock to be seamless.
- ✚ The ease of data transfer to partners if needed (i.e. The Export of order details for logistics operations)
- ✚ The ability to generate compiled Analytics about said transactions across its threads in a user-friendly format (.CSV), allowing the information to be imported into Microsoft Excel and Microsoft Power BI with ease.

These in turn, increases efficiency for our Client, allowing freed-up resources to be allocated elsewhere.

Implementation Scope

Adding Products

- ✚ Supported the importing of large quantities of products as well as the respective stock levels for them from an external source (.CSV file). These products are then stored in the Central Inventory System Data Structure.
- ✚ A standard format (see System Overview & Architecture for further details) for .CSV files was created for imports
- ✚ Supported the adding of products and their stock level manually to the System via the GUI

Removing Products

- ✚ Supported the adding of products and their stock levels manually to the System via the GUI

Displaying of all products.

- ✚ Display of all current products contained within the system, along with respective stock count via our CLI/GUI.

Searching Products

- ✚ Supported the searching of products. Users have the option to search via the product name or via the Product_ID.
- ✚ The search option is available to the user via the CLI/GUI, allowing the user to input a Product_ID or Name. When a match is found, it returns the Product_ID and the amount of stock on hand. When match is not found, the System returns a “No Matches Found” and returns to the Main Menu.

Sorting Products

- ✚ Supported the sorting of products in multiple ways
- ✚ Products can be sorted by Price, Quantity, Alphabetically and by Category.

Concurrent Order Processing

- ✚ Multi-threading was used to support the simultaneous order processing across the warehouses.
- ✚ Thread count can be set by the user, allowing the system to be optimally configured for different systems.

Loading Orders into the System

- ✚ Supported the manual loading of Orders from warehouses a .CSV file. A standardised format is used (see System Overview & Architecture for further details) across all warehouses.

System Analytics

- ✚ Thread logs are compiled and fed into an Analytics Manager for the System.
- ✚ Tracked items include: Un/successful orders, Total orders, revenue earned.
- ✚ A standardised format is used for .CSV allowing the ease of export when needed. This can then be imported into Microsoft Excel and Microsoft Power BI for data representation.

Product Catalogue Export

- ✚ When the program is closed, the current Inventory (Products and Stock) should be automatically exported to a .CSV file to be used when the program is launched again.
- ✚ Supported the export of the “current” state of the Centralised Inventory System (Product information and their quantities) as a standardised .CSV file which is then read when the system is run again. This allows the Centralised Inventory System to “save” itself.

Summary of Key features Delivered.

- ✚ Stable support for Multi-Threading up to 32 Threads (The maximum number of Threads is variable and can be configured for any number of Threads if needed, however, we were only able to test up to 32 threads).
 - ✚ Support for 5 Warehouses as inputs, with the necessary scalability in place to support more in the future.
 - ✚ The parsing in Products and Orders via .CSVs making it easier to support connection to a database if needed.
 - ✚ Support for the Adding, Removing and Editing product information via the CLI/GUI as well as Searching/Sorting through them.
 - ✚ Thorough detailed reports of System Analytics which is available as exports for Data processing Applications like Excel and PowerBI.
 - ✚ Inventory System Persistence when the system is ShutDown and relaunched.
 - ✚ A comprehensive CLI as well as a GUI to maintain functionality for resource limited systems as well as provide an intuitive, user-friendly interface for more advanced environments, ensuring accessibility, flexibility, and scalability across a wide range of user needs and hardware capabilities.
-

System Architecture

MULTI-THREADED REAL-TIME INVENTORY & TRADING SYSTEM PROCESS OVERVIEW

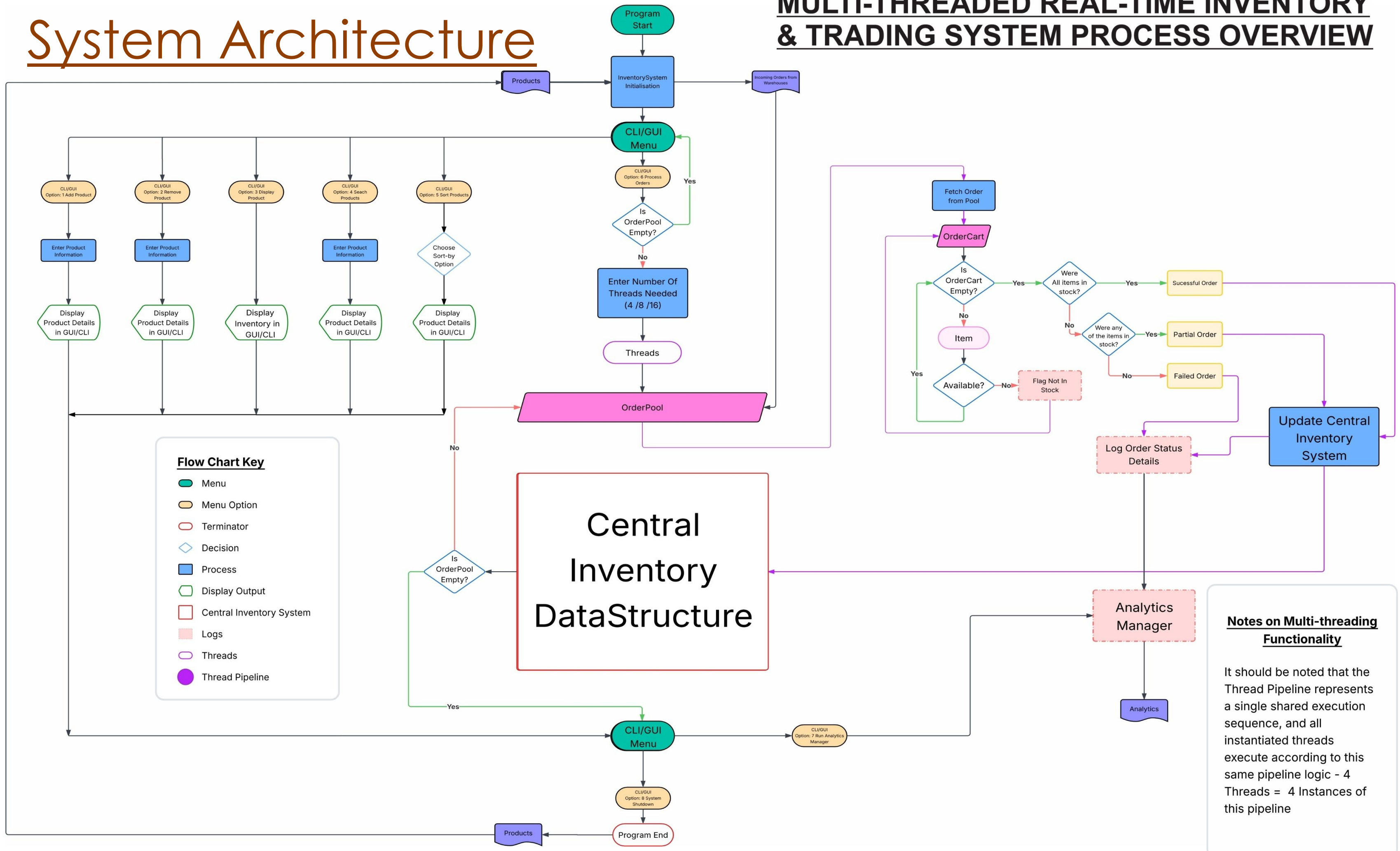


Figure 1: A Flowchart detailing the flow of data through the System as well as how components interact with each other

MULTI-THREADED REAL-TIME INVENTORY & TRADING SYSTEM PROCESS OVERVIEW

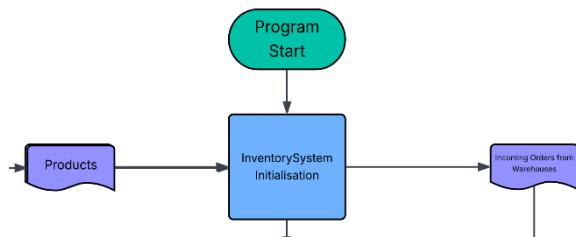


Figure 2 A diagram detailing the startup of the program

Central Inventory System and Order Pool

Central Inventory System

When the program is launched, the InventorySystem first needs to initialise itself. The initialisation begins with the Products.CSV file. This file contains all the Products that's to be stored within the InventorySystem as well as the current quantity that we will have on hand at the start of the program. The idea is to accept Products.CSV, which contains products in the following form:

ProductID,Name,Category,Price,QTY,TYPE,TaxRate,DiscountRate

*(It should be noted that TaxRate and DiscountRate is dependent on the TYPE. Where its not applicable, the entry remains as 0. – Example, a Taxable Product will have 0 for its DiscountRate entry. *Product Types to be Explained later on)*

733	Soil Sieve 1/4" Mesh 12"	Garden	336	259	T	0.15	0
734	Garlic Powder 3oz	Perishables	144	261	S	0	0
735	Scan Tool Professional	Automotive	1922	478	D	0	0.13

Figure 3 An example of the structure of the Products.CSV taken from Excel

The CSV is passed Directly in to the InventorySystem, where it takes the above information and turns it into an "Object" stored within. Once initialisation has completed and all products are loaded into the InventorySystem, the user may now access the GUI/CLI part of the program.

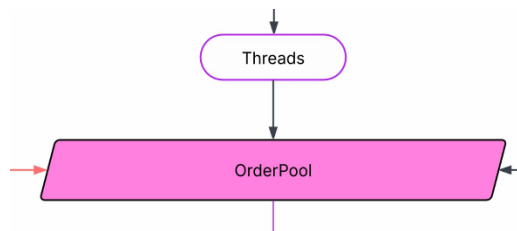


Figure 4 The OrderPool using an STL Queue

The OrderPool

The OrderPool serves as a container to hold our orders before they are processed via the threads. The idea is that the instantiated threads access the OrderPool and Retrieve an Order Object that is to be processed by the respective thread. Backlog Orders accumulated while the system is offline (Like on a Public Holiday/Sunday) are loaded in to the OrderPool at start up from a WarehouseN_Orders.csv (Where N is each Warehouse number). Each entry in the CSV is a representation of an Order cart with Products and the quantity wanted:

ProductID,QTYRecurring**

Order Carts can contain Multiple Products at a time, hence the use of **Recurring. Order Objects are then created and placed within the OrderPool (Details to follow). Here is an example of an order from a warehouse that has an order with 2 Products:

27	4	33	5
----	---	----	---

Figure 5 An example of the Order structure of the WarehouseN_Orders.csv taken from Excel. The displayed entry contains 2 Products in the order cart with the trailing quantity requested – ProductID,QTY,ProductID,QTY

Thread Pipeline

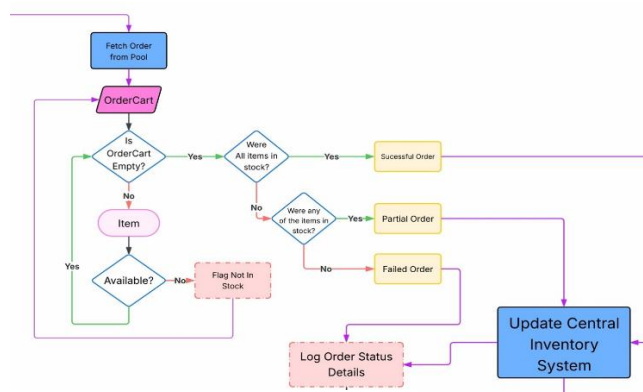


Figure 6 Overview of the Thread Pipeline

Once the system detects that the Cart/Basket is empty, it needs to decide on how to Mark the order. If all items were in stock its marked as *Successful*. If there was an item that was not in stock, it will check if all items weren't in stock then mark it as *Failed*, if some items were in stock its marked as *Partial*. Once the checks are performed the Central Inventory System (if Successful/Partial) is updated and a Log of the order (AnalyticsRecord) is created which is sent to the Analytics Manager (Analytics details will be explained in the sections to be followed).

The threads are instantiated onto the OrderPool, where they received their order objects to be Processed. Each thread will be an instance of the pipeline shown on the left. Order Objects are fetched from the OrderPool. This is where concurrency starts and how shared data is accessed. Since the Order Object is a "Cart/Basket of items", it loops through each item until there are no items left in the Cart/Basket. When An item is retrieved from the Cart/Basket, the

CLI Overview and System Flow

Once the User has access to the CLI after initilisation (See the start of the section) , they will be presented with the following Options:

1. Add a new product – *(The user follows the prompts to enter a new Product)*
2. Remove a product – *(The user searches for a product can remove it when found)*
3. Edit a product - *(Search for a Product, then Prompted to Edit Product Information)*
4. Display all Products – *(ALL Entries with the Central Inventory System is Displayed)*
5. Search - *(Search via name or ProductID)*
6. Sort Products - *(Sort by Price, Quantity, Type, Alphabetically and Categorically)*
7. Process Orders (Multi-Threaded)*
8. Analytics*
9. Save & Shutdown – *(Shuts Down and saves its Inventory for when its launched again)*

Note. All those marked with a "*" will be explained in this section. For the rest, please refer to the Video demonstration that will detail how the User interacts with these functions.

Process Orders

When this option is selected. The user enters the number of Threads they would like to have. Once entered, the system starts to process all Orders in the OrderPool simultaneously via the Thread pipeline described above. Thread Logs and Order Details are displayed on the screen along with the Status of the Order. Once completed, a quick summary is displayed of the batch that was processed. Shared Data is accessed by the Threads in the OrderPool and Inventory System. All relevant concurrency logic has been implemented to prevent unwanted behavior (See Concurrency Design and Thread Safety for more information).

Analytics

This option controls all Analytics of the System. The User can Generate individual Warehouse Analytics Reports, A Global Analytics Report as well as export all of these to a .CSV. The user has the choice to select exactly what they would like exported.

System Data Flow

The system follows a structured flow of data between its core components:

1. The system is initialised by loading product data from Products.csv into the Inventory System.
2. Order data is loaded from warehouse CSV files into the OrderPool as Order Objects.
3. When the user selects *Process Orders*, multiple threads are created.
4. Each thread retrieves an Order from the OrderPool.
5. The thread processes each item in the order:
 - ✚ Checks stock availability in the Inventory System
 - ✚ Updates stock if available
6. Once processing is complete:
 - ✚ The order is marked as Successful, Partial, or Failed
 - ✚ An AnalyticsRecord is generated
7. The Analytics Manager stores the results
8. Results are displayed to the user via CLI/GUI

Concurrency and Shared Data Access

Concurrency is introduced during the order processing phase, where multiple threads operate simultaneously on shared resources.

The following shared components are accessed concurrently:

- ✚ **OrderPool:** Threads retrieve orders from a shared queue structure
- ✚ **Inventory System:** Threads read and update product quantities

To ensure thread safety and prevent race conditions:

- ✚ Synchronisation mechanisms (e.g., mutex locks) are used
- ✚ Only one thread can modify critical data at a time
- ✚ Read/write operations are controlled to maintain data consistency

Class Design & OOP Structure

MULTI-THREADED REAL-TIME INVENTORY & TRADING SYSTEM UML DIAGRAM

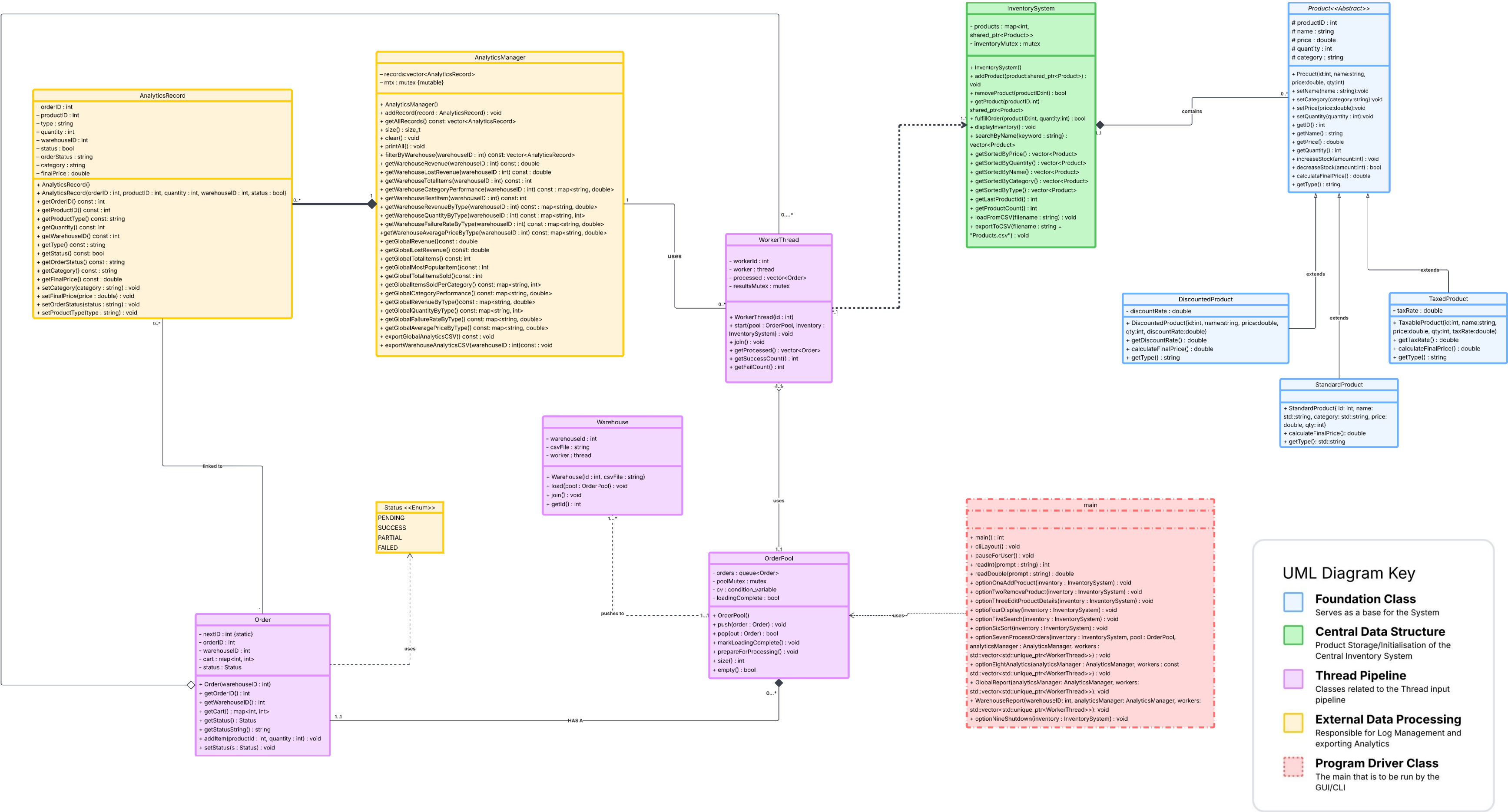


Figure 7 A UML Class Diagram Overview showcasing various Classes in the System

Overview

The system is designed using object-oriented principles to ensure modularity, scalability, and maintainability. The architecture is centred around a product-based hierarchy, a thread-safe order processing pipeline, and a dedicated analytics subsystem.

The relationships between classes, including inheritance and composition, are illustrated in the UML diagram as seen above.

Object Oriented Design Principles

Encapsulation

Encapsulation is achieved by restricting direct access to class attributes and exposing controlled access through public methods. For example, product attributes such as price, quantity, and category are accessed and modified through setter and getter methods. This prevents invalid modifications and ensures data integrity.

Abstraction

Abstraction is implemented through the use of the abstract Product base class. This class defines a general interface for all product types while hiding implementation details. Each derived class provides its own implementation of behaviour such as pricing logic.

Inheritance

Inheritance is used to extend the Product base class into specialised product types. This allows shared attributes (e.g., ID, name, price, quantity) to be reused while enabling different pricing behaviours in derived classes.

Polymorphism

Polymorphism is achieved through the use of base class pointers (`shared_ptr<Product>`). This allows the system to treat all product types uniformly while invoking the correct overridden methods at runtime based on the actual object type.

Composition

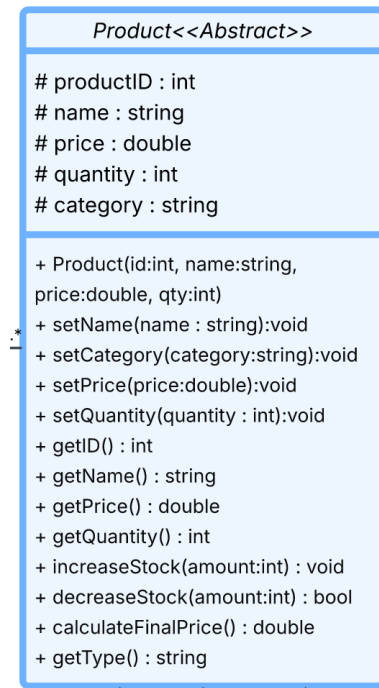
Composition is used in the analytics subsystem. The AnalyticsManager contains and manages multiple AnalyticsRecord objects, forming a strong ownership relationship where records exist only within the manager.

Concurrency Design

The system uses multi-threading to process orders concurrently. Shared resources such as the InventorySystem and OrderPool are protected using mutexes and condition variables to prevent race conditions and ensure thread safety.

Major Class Responsibilities

Product (Abstract Base Class)



The Product class defines the core structure for all product types. It includes attributes such as:

- Product ID
- Name
- Price
- Quantity
- Category

It declares the following virtual functions:

- calculateFinalPrice()
- getType()

These enforce that all derived classes implement their own pricing logic and type identification.

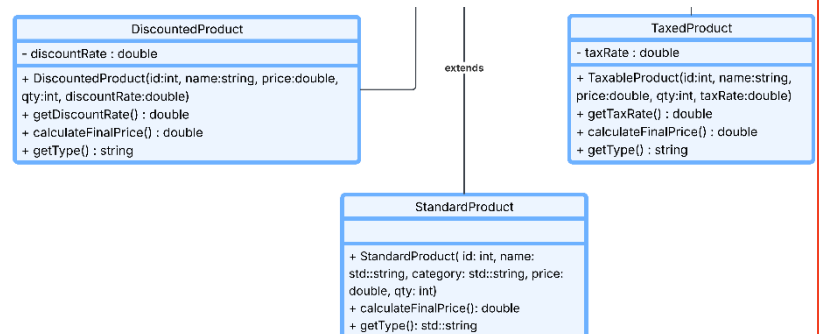


Figure 8 The Product Base Class

Derived Product Classes

TaxableProduct: Applies a tax rate to the base price when calculating the final price

DiscountedProduct: Applies a discount rate to reduce the base price. The Standard Tax rate (0.15) is included in its overridden calculatePrice().

Figure 9 Derived Classes from the base Product Class

StandardProduct: Represents products with no tax or discount applied. This class exists because the base Product class is abstract and cannot be instantiated directly. This allows us to have Zero-rated (Tax Free) products in our System.

All derived classes override:

- calculateFinalPrice()
- getType()

InventorySystem

The InventorySystem is responsible for managing all products in the system.

Class Responsibilities:

- Storing products using shared_ptr<Product>
- Providing search and sorting functionality
- Updating stock levels during order processing
- Ensuring thread-safe access using mutex locks

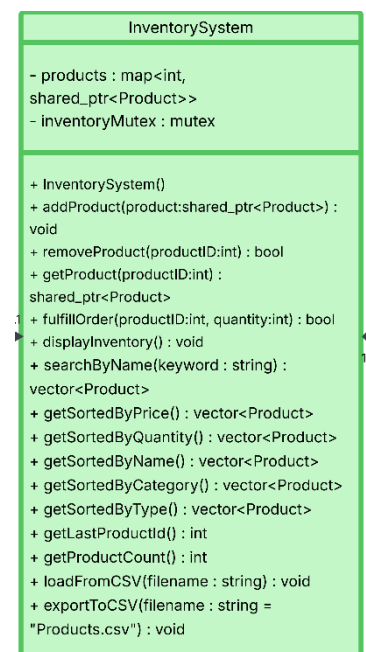


Figure 10 The Inventory System Class

Order

The Order class represents a collection of items requested by a warehouse.

This is known as a “Cart/Basket”

Key features:

- Stores multiple products and quantities in a Map
- Uses a static atomic counter to generate unique order IDs
- Supports safe usage in a multi-threaded environment

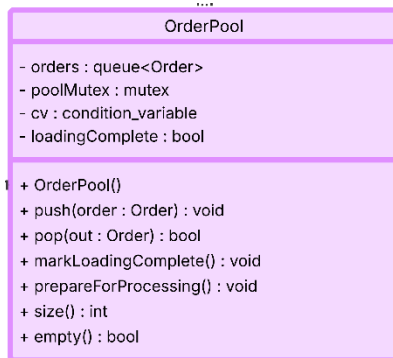


Figure 12 The OrderPool Class

OrderPool

The OrderPool acts as a shared queue of orders waiting to be processed.

Class Responsibilities:

- Thread-safe insertion and retrieval of orders
- Synchronisation using mutexes and condition variables
- Coordination between producer threads (warehouses) and consumer threads (workers)

AnalyticsRecord

The AnalyticsRecord class is a data container that stores item-level transaction data generated during order processing.

This Class Logs the Following Details within its Object:

- Product details
- Quantity
- Warehouse ID
- Final price
- Order status



Figure 14 The AnalyticsManager Class

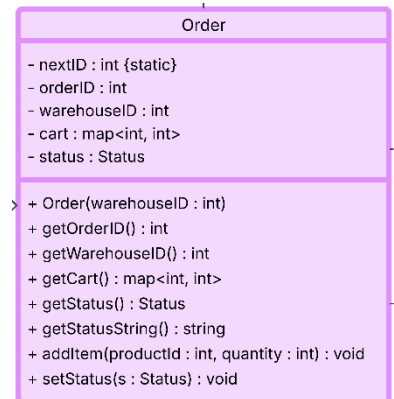


Figure 11 The Order Class

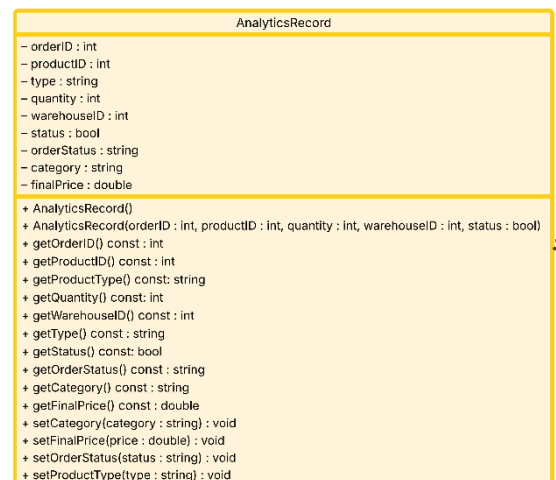


Figure 13 The AnalyticsRecord Class

AnalyticsManager

The AnalyticsManager processes and aggregates analytics data.

Class Responsibilities:

- Storing AnalyticsRecord objects
- Generating warehouse-level and global reports
- Calculating revenue, success rates, and performance metrics
- Exporting analytics data to CSV format

CLI / Main System

The CLI acts as the control layer of the system.

Responsibilities:

- ✚ Handling user interaction
- ✚ Managing product operations (add, remove, edit, search, sort)
- ✚ Triggering multi-threaded order processing
- ✚ Displaying analytics results

```
main
+ main() : int
+ cliLayout() : void
+ pauseForUser() : void
+ readInt(prompt : string) : int
+ readDouble(prompt : string) : double
+ optionOneAddProduct(inventory : InventorySystem) : void
+ optionTwoRemoveProduct(inventory : InventorySystem) : void
+ optionThreeEditProductDetails(inventory : InventorySystem) : void
+ optionFourDisplay(inventory : InventorySystem) : void
+ optionFiveSearch(inventory : InventorySystem) : void
+ optionSixSort(inventory : InventorySystem) : void
+ optionSevenProcessOrders(inventory : InventorySystem, pool : OrderPool, analyticsManager : AnalyticsManager, workers : const std::vector<std::unique_ptr<WorkerThread>>) : void
+ optionEightAnalytics(analyticsManager : AnalyticsManager, workers : const std::vector<std::unique_ptr<WorkerThread>>) : void
+ GlobalReport(analyticsManager : AnalyticsManager, workers : const std::vector<std::unique_ptr<WorkerThread>>) : void
+ WarehouseReport(warehouseID : int, analyticsManager : AnalyticsManager, workers : const std::vector<std::unique_ptr<WorkerThread>>) : void
+ optionNineShutdown(inventory : InventorySystem) : void
```

Figure 15 The Main

Base Class Design and Derived Behaviour

The Product class serves as the foundation of the system's design. By defining a common interface and shared attributes, it allows all product types to be handled uniformly.

Derived classes extend this base structure by implementing their own pricing logic. For example:

- ✚ TaxableProduct increases price using tax
- ✚ DiscountedProduct reduces price using a discount
- ✚ StandardProduct applies no modification

This design ensures flexibility while maintaining a consistent structure across all product types.

Justification for Virtual Functions

Virtual functions are essential in the Product class to enable runtime polymorphism. Since different product types require different pricing logic, the system cannot determine the correct method implementation at compile time.

By declaring calculateFinalPrice() and getType() as virtual functions, the system ensures that the correct method is called based on the actual object type, even when accessed through a base class pointer (shared_ptr<Product>).

This approach allows:

- ✚ Uniform handling of all product types
- ✚ Flexibility to introduce new product types
- ✚ Clean separation of behaviour across classes

Conclusion

The system is built around a modular, object-oriented design that integrates inheritance, polymorphism, and concurrency. The abstract Product class defines a consistent structure for all product types, while derived classes implement specialised behaviour.

Thread-safe components such as the OrderPool and InventorySystem enable concurrent order processing without compromising data integrity. The analytics subsystem further enhances the system by providing detailed insights into system performance.

Overall, the design ensures scalability, maintainability, and efficient execution, making it suitable for high-performance, multi-threaded environments.

Data Structures & STL Usage

Overview

The system leverages the C++ Standard Template Library (STL) to provide efficient, scalable, and thread-safe data management. The selection of STL containers and algorithms was guided by performance requirements, memory safety, and suitability for concurrent execution.

Containers Used and Justification

[std::map<int, std::shared_ptr<Product>> \(Inventory System\)](#)

The central inventory is implemented as:

```
std::map<int, std::shared_ptr<Product>> products;
```

This structure maps a unique ProductID to a Product object.

Justification:

- ✚ Guarantees unique keys, preventing duplicate products
- ✚ Provides $O(\log n)$ complexity for insertion, lookup, and deletion
- ✚ Enables efficient product retrieval using `products.find(productId)`
- ✚ Maintains sorted order, allowing access to highest ID via `rbegin()`

```
private:
    std::map<int, std::shared_ptr<Product>> products;
    std::mutex inventoryMutex;

public:
    void addProduct(std::shared_ptr<Product> product);
    bool removeProduct(int productId);
    std::shared_ptr<Product> getProduct(int productId);
```

Figure 16 An Extract from our InventorySystem header file showing how the products data structure is used together with shared_ptrs and mutex

The use of `std::shared_ptr<Product>` ensures:

- ✚ Safe memory sharing across multiple components (Inventory, Workers, CLI)
- ✚ Automatic deallocation when no references remain
- ✚ Elimination of manual memory management errors

[std::queue<Order> \(OrderPool\)](#)

The order processing pipeline uses:

```
std::queue<Order> orders;
```

Justification:

- ✚ Implements FIFO ordering, ensuring fairness in processing
- ✚ Provides $O(1)$ enqueue and dequeue operations
- ✚ Ideal for producer-consumer concurrency models

```
//Thread-safe order queue shared between producers (warehouses) and consumers (workers)
//Uses condition_variable so workers sleep instead of busy-waiting (wastes CPU)
class OrderPool {
private:
    std::queue<Order> orders;
    mutable std::mutex poolMutex; //Have to use mutable to lock const functions
    std::condition_variable cv;

    //When true, workers know no more orders are coming
    bool loadingComplete;
```

The queue is combined with a condition variable and Mutex to ensure thread safety and efficiency.

Figure 17 An Extract of a Queue from our OrderPool Header

std::map<int, int> (Order Cart)

Each order stores its items using:

std::map<int, int> cart;

Justification:

- ✚ Maps ProductID → Quantity efficiently
- ✚ Prevents duplicate entries within the same order
- ✚ Allows fast updates and lookups during processing
- ✚ Maintains consistency with the inventory structure

```
//Atomic so multiple warehouse threads can safely create orders at the same time
static std::atomic<int> nextID;

int orderID;
int warehouseID;
std::map<int, int> cart; //productId -> quantity
Status status;
```

Figure 18 An Extract from our Order Class Header

std::vector<std::shared_ptr<Product>> (Snapshots & Sorting)

Used in multiple places, e.g.:

std::vector<std::shared_ptr<Product>> snapshot;

std::vector<std::shared_ptr<Product>> v;

Justification:

- ✚ Provides contiguous memory, improving cache performance
- ✚ Enables efficient iteration and transformation
- ✚ Required for use with STL algorithms like std::sort

```
//Uses find_if to search by name (partial match)
std::vector<std::shared_ptr<Product>> searchByName(const std::string& keyword);

//Sorting, returns a copy so the original map is not affected
std::vector<std::shared_ptr<Product>> getSortedByPrice();
std::vector<std::shared_ptr<Product>> getSortedByQuantity();
std::vector<std::shared_ptr<Product>> getSortedByName();
std::vector<std::shared_ptr<Product>> getSortedByCategory();
std::vector<std::shared_ptr<Product>> getSortedByType();
```

Figure 19 An Extract From our InventorySystem Header

std::vector<std::unique_ptr<WorkerThread>> (Thread Management)

std::vector<std::unique_ptr<WorkerThread>> workers;

Justification:

- ✚ Allows dynamic creation of worker threads
- ✚ Supports flexible scaling (user-defined thread count)
- ✚ Ensures exclusive ownership via unique_ptr

Benefits:

- ✚ Prevents accidental sharing of thread objects
- ✚ Automatic cleanup when threads are removed

std::vector<AnalyticsRecord> (Analytics Storage)

std::vector<AnalyticsRecord> records;

Justification:

- ✚ Efficient for sequential data storage
- ✚ Optimised for iteration during analytics computation
- ✚ Low overhead compared to associative containers

STL Algorithms Used

std::find_if (Searching)

Used to perform flexible searches based on conditions (e.g., partial name match)

Why it was chosen

- ✚ Improves readability compared to manual loops.
- ✚ Allows lambda-based conditions.
- ✚ More expressive and maintainable.

Performance

- ✚ Linear time complexity: $O(n)$
- ✚ Appropriate since search is not key-based.

```
//Uses std::find_if to do a partial name match (case-sensitive)
//find_if scans linearly which makes sense here since map is keyed by ID not name
std::vector<std::shared_ptr<Product>> InventorySystem::searchByName(const std::string& keyword) {
    std::lock_guard<std::mutex> lock(inventoryMutex);
    std::vector<std::shared_ptr<Product>> results;

    auto it = products.begin();
    while (it != products.end()) {
        //find_if returns the first match from current position onward
        it = std::find_if(it, products.end(),
            [&keyword](const std::pair<const int, std::shared_ptr<Product>>& entry) {
                return entry.second->getName().find(keyword) != std::string::npos;
            });

        if (it != products.end()) {
            results.push_back(it->second);
            ++it; //Move past this match to keep searching
        }
    }
    return results;
}
```

Figure 20 An extract from our InventorySystem that uses find_if

std::sort (Sorting)

Used to sort products based on different attributes.

Justification:

- ✚ Highly optimised
- ✚ Time complexity: $O(n \log n)$
- ✚ Supports custom comparison using lambda functions
- ✚ Eliminates need for manual sorting algorithms

```
std::vector<std::shared_ptr<Product>> InventorySystem::getSortedByPrice() {
    std::lock_guard<std::mutex> lock(inventoryMutex);
    std::vector<std::shared_ptr<Product>> v;
    for (const auto& pair : products) v.push_back(pair.second);
    std::sort(v.begin(), v.end(),
        [](const std::shared_ptr<Product>& a, const std::shared_ptr<Product>& b) {
            return a->calculateFinalPrice() < b->calculateFinalPrice();
        });
    return v;
}
```

Figure 21 An Extract from our InventorySystem showing how one of the sorting functions use std::sort

Used for sorting by:

- ✚ Price
- ✚ Quantity
- ✚ Name
- ✚ Category
- ✚ Type

Summary

The system makes effective use of STL containers and algorithms to achieve:

- ✚ **Efficiency:** Logarithmic and constant-time operations where needed for maximum Performance
- ✚ **Safety:** Smart pointers and mutex protection prevent memory and threading issues
- ✚ **Scalability:** Dynamic containers allow easy expansion
- ✚ **Maintainability:** STL algorithms reduce complexity and improve readability

By leveraging std::map, std::vector, std::queue, and STL algorithms like std::find_if and std::sort, the system achieves a balance between performance, safety, and clean design—making it well-suited for a high-performance multi-threaded inventory and order processing environment.

Concurrency Design & Thread Safety

This section explains how the Multi-Threaded Inventory and Order Processing System implements concurrency, identifies every shared resource that crosses thread boundaries, and details the synchronisation primitives that keep the system free from race conditions and deadlocks. The design follows a producer–consumer model: warehouses generate orders, worker threads consume them, and a central inventory acts as the shared state both sides interact with.

Thread Model and Roles

The system runs in two clearly separated phases, each with its own pool of threads. Phase 1 (Loading) launches five warehouse threads in parallel via the Warehouse class. Each reads its assigned CSV file (Warehouse1_Orders.csv ... Warehouse5_Orders.csv), parses every line into an Order object, and pushes it into the shared OrderPool. Phase 2 (Processing) launches a user-selected number of WorkerThreads — typically 4, 8, or 16 — so the system scales to the host hardware. Each worker repeatedly pops an order, fulfils every cart item against the inventory, classifies the order as SUCCESS, PARTIAL, or FAILED, and writes a record into the AnalyticsManager.

The two phases are temporally separated: Phase 1 finishes before Phase 2 begins, so warehouse and worker threads never run simultaneously. The pool's loadingComplete flag is reset by prepareForProcessing() between phases so workers do not see stale state. Active threads at any moment range from five during loading to up to thirty-two during processing, plus the main thread driving the CLI.

Shared Resources and Synchronisation Strategy

Every piece of state crossing thread boundaries was identified during design and given an explicit synchronisation tool, summarised below

Shared Resource	Class / Location	Synchronisation Tool	Risk Without It
Order ID counter	Order::nextID	<code>std::atomic<int></code>	Duplicate Order IDs
Order queue	OrderPool::orders	<code>std::mutex + condition variable</code>	Torn queue, busy-wait
Inventory map	InventorySystem::products	<code>std::lock_guard<std::mutex></code>	Negative stock
Analytics records	AnalyticsManager::records	<code>std::lock_guard<std::mutex></code>	Vector reallocation crash
Per-worker results	WorkerThread::processed	<code>std::lock_guard<std::mutex></code>	Read-while-write race
Console output	Logger.h printMutex	<code>std::mutex</code> via LOG macro	Interleaved output

OrderPool — std::mutex with a Condition Variable

The OrderPool wraps a `std::queue<Order>` shared between five producers and up to thirty-two consumers. A plain mutex would force workers to busy-wait on an empty queue, wasting CPU. The pool therefore pairs `poolMutex` with `std::condition_variable` `cv`. Producers push under the lock and call `cv.notify_one()`; consumers acquire a `std::unique_lock` (required because `condition_variable` releases the lock while

```
bool OrderPool::pop(Order& out) {
    std::unique_lock<std::mutex> lock(poolMutex);

    //Sleep until there's an order or we know no more are coming
    cv.wait(lock, [this] {
        return !orders.empty() || loadingComplete;
    });

    //Pool is done and empty, worker should stop
    if (orders.empty())
        return false;

    out = orders.front();
    orders.pop();
    return true;
}
```

Figure 22 An extract from the OrderPool Class

sleeping) and call `cv.wait()` with a predicate. The predicate is critical — it protects against spurious wake-ups and gives workers a clean exit path when `markLoadingComplete()` sets the flag and `notify_all()` wakes every sleeper.

InventorySystem — Coarse-Grained Mutex

The `InventorySystem` owns the `std::map<int, std::shared_ptr<Product>>` holding every product. STL containers are not thread-safe, so a single `inventoryMutex` guards every public method. `fulfillOrder()` is the most performance-critical because every worker calls it on every cart item — it locks, looks up the product, calls `decreaseStock(quantity)`, and returns whether stock was sufficient, all atomically.

```
//Lock ensures two workers cant reduce the same stock at the same time
bool InventorySystem::fulfillOrder(int productId, int quantity) {
    std::lock_guard<std::mutex> lock(inventoryMutex);
    auto it = products.find(productId);
    if (it != products.end())
        return it->second->decreaseStock(quantity);
    return false;
}
```

Figure 23 An extract from the `InventorySystem` Class

A coarse-grained mutex was chosen over per-product locks because it makes the read-modify-write of stock atomic by construction (two workers cannot both pass a stock check and then both decrement, the classic race that causes negative stock), and the critical section is short enough that contention is acceptable even with sixteen workers. Slow operations like `displayInventory()` copy product pointers under the lock and release it before printing, keeping the critical section bounded.

AnalyticsManager, WorkerThread Buffers, and Console Output

The `AnalyticsManager` collects an `AnalyticsRecord` for every cart item and protects its internal vector with a mutable `std::mutex` (mutable so `const` accessors can still acquire it). Each `WorkerThread` also guards its private processed vector with a `resultsMutex`, preventing the main thread from reading it while the worker is still appending. Both use `std::lock_guard` exclusively — its RAII semantics guarantee the mutex releases even if an exception is thrown. Console output is serialised through a global `printMutex` declared inline in `Logger.h`; without it, `std::cout` calls from sixteen workers would shred lines together. Logging once per order rather than per cart item keeps the print mutex from becoming a serialising bottleneck.

Order ID Generation — std::atomic

Order IDs must be unique across all warehouse threads constructing `Order` objects concurrently. Wrapping the increment in a mutex would work but is heavier than necessary for a single integer. The `Order` class uses `std::atomic<int> nextID` with `nextID.fetch_add(1)`, which is lock-free on every modern platform and guarantees no two orders share an ID

Race Conditions, Failures, and Evidence

Three concrete races were identified during design and eliminated by placing the read-modify-write sequence inside the same critical section:

- ✚ Stock decrement race — two workers ordering the last unit of a product. Both reads see stock = 1, both pass the check, both decrement. Eliminated by holding `inventoryMutex` across the entire `fulfillOrder()` body.
- ✚ Order ID race — two warehouse threads reading the same `nextID`. Eliminated by `atomic fetch_add`, a single uninterruptible CPU instruction.
- ✚ Queue corruption race — concurrent push/pop on `std::queue` corrupts internal pointers. Eliminated by routing every queue operation through `poolMutex`.

Deadlocks are prevented by a simple rule the design enforces: no thread ever holds two mutexes simultaneously. Each public method acquires at most one lock, so the cyclic-wait condition required for deadlock cannot form. Order failures are not exceptional — they are an expected outcome whenever quantity exceeds stock. Inside the worker loop, successCount and failCount are tallied locally; afterwards a single classification step assigns the final status, so a failed item never partially decrements stock and exceptions never cross thread boundaries.

```
if      (failCount == 0)    order.setStatus(Order::Status::SUCCESS);  
else if (successCount == 0) order.setStatus(Order::Status::FAILED);  
else                                     order.setStatus(Order::Status::PARTIAL);
```

Figure 24 An extract from the WorkerThread where statuses are given

Concurrency is verified at runtime by inspecting the interleaved log output. Every worker prints its workerId and std::this_thread::get_id() with each completed order, producing lines that visibly interleave from different OS thread IDs:

```
[Worker 8 | Thread 14] Order #1097 | Warehouse 3 | Items: 1 | SUCCESS  
[Worker 24 | Thread 30] Order #1095 | Warehouse 2 | Items: 4 | FAILED  
[Worker 9 | Thread 15] Order #1099 | Warehouse 2 | Items: 2 | PARTIAL
```

Figure 25 An extract of the Console showing the Orders being processed

Scalability was confirmed empirically: the same dataset processed with increasing worker counts shows wall-clock time decreasing, then plateauing once the inventory mutex becomes the bottleneck. Final inventory totals after every run match the expected closing stock exactly — the strongest practical evidence that no updates are lost and stock never drops below zero.

Memory Management Strategy

This section explains how the Multi-Threaded Inventory and Order Processing System manages memory safely and efficiently using modern C++ techniques. The design centres on smart pointers, clear ownership semantics, and automatic resource management, ensuring that dynamically allocated objects are correctly managed without leaks or undefined behaviour. Particular attention is given to how memory management integrates with the system's concurrent design, where predictable object lifetimes are essential.

It should be noted that, with regards to the design, is that memory management is tightly integrated with concurrency. Since multiple worker threads operate on shared inventory and analytics structures, predictable object lifetimes and strict ownership rules are essential to prevent undefined behaviour, memory corruption, or lifetime-related race conditions.

Smart Pointer Usage and Ownership Model

We have selected to use smart pointers across our system. All dynamically allocated objects in the system are managed using smart pointers rather than raw pointers. This removes the need for manual memory deallocation and ensures that object lifetimes are automatically tied to scope and ownership.

Product objects within the InventorySystem are stored using smart pointers inside a standard container:

```
std::map<int, std::shared_ptr<Product>> products;
```

This design allows multiple system components (such as worker threads and Analytics) to safely reference the same product data without duplicating ownership. The inventory remains the single logical owner of product lifetimes, while shared references ensure safe concurrent access.

Worker threads themselves are managed using exclusive ownership:

```
std::vector<std::unique_ptr<WorkerThread>> workers;
```

Each thread instance has a single owner, typically the controlling thread or thread manager. Once the container goes out of scope, all worker threads are automatically cleaned up without requiring explicit deletion.

This creates a clear and consistent ownership model:

-  Inventory owns product lifetimes
-  Thread manager owns worker thread lifetimes
-  Shared access is allowed only where explicitly required

[unique_ptr vs shared_ptr](#)

The system distinguishes between `std::unique_ptr` and `std::shared_ptr`, as they represent different ownership strategies.

std::unique_ptr enforces exclusive ownership of a resource. It cannot be copied, only moved, which guarantees that exactly one owner exists at any time. This makes it lightweight and ideal for objects such as worker threads, where ownership transfer is well-defined and duplication is unnecessary

In contrast, ***std::shared_ptr*** allows multiple parts of the system to reference the same object through a reference-counting mechanism. The object is destroyed only when the final reference is released. While this enables shared access, it introduces overhead and requires careful use.

Therefore, in the system:

- ✚ ***std::unique_ptr*** is used for single-owner resources such as worker threads and thread controllers
- ✚ ***std::shared_ptr*** is used for shared domain objects such as products stored in the inventory

The choice of pointer type is therefore based on ownership requirements rather than convenience.

Memory Safety and Leak Prevention

Memory safety is guaranteed through consistent use of RAI (Resource Acquisition Is Initialization). All dynamically allocated objects are bound to smart pointers, meaning they are automatically destroyed when their owning scope ends. This removes the need for manual delete operations and eliminates common memory-related errors.

Specifically, the design prevents:

- ✚ Memory leaks, since all allocations are owned by smart pointers
- ✚ Dangling pointers, since objects cannot outlive their owners
- ✚ Double deletion, since ownership is strictly enforced and non-duplicable in ***unique_ptr***

Even in error conditions or early exits from worker loops, this ensures that resources are released correctly without requiring explicit cleanup logic.

Integration with Concurrent Execution

Memory management is closely aligned with the system's concurrency model. While smart pointers handle ownership and lifetime, mutexes and condition variables ensure safe concurrent access to shared memory regions.

For example, the InventorySystem protects its internal product map using a mutex:

```
std::lock_guard<std::mutex> lock(inventoryMutex);
```

This separation ensures that:

- ✚ Smart pointers manage lifetime and ownership
- ✚ Mutexes manage safe concurrent access

Additionally, longer operations such as display or reporting work on temporary copies of data where appropriate. This allows locks to be released early, reducing contention and improving overall system responsiveness.

Summary

The memory management strategy is built on three core principles: controlled object creation, explicit ownership, and automatic resource cleanup. By using ***std::unique_ptr*** for exclusive ownership and ***std::shared_ptr*** for controlled shared access, the system ensures that every object has a well-defined lifecycle. Combined with mutex-protected shared state, this guarantees that memory remains safe, consistent, and leak-free even under heavy multi-threaded execution. Overall, this approach eliminates traditional pointer-related errors while supporting a scalable and thread-safe processing architecture.

User Interface Design.

CLI Structure and Navigation

The system uses a central main menu as the only entry point to all functionality. Users interact using numeric input to access as seen on the left. Each operation executes and returns to the main menu, ensuring a controlled interaction cycle and preventing overlapping execution states. It should be noted that inputs are sanitised accordingly. Below is an example of how option one will play out.

```
Enter option: 1
||----- Select Product Type -----||
1. Standard (Zero-Rated Item - Perishables ONLY)
2. Taxable
3. Discounted
Choice: 2
Name: FinalReportScreenshot
||----- Select Category -----||
1. Electronics
2. Perishables
3. Clothing
4. Home
5. Sports
6. Toys
7. Automotive
8. Books
9. Beauty
10. Garden
11. Office
Category Choice: 11
Price: R1
Quantity: 1
Tax rate (e.g. 0.15): 0.15
Product Added | ID=1001 | FinalReportScreenshot | Office | Stock=1
Press ENTER to return to menu...|
```

Figure 28 An example of System Flow when the user selects Option 1

ID: 1	Name: Gloves Touchscreen Compatible	Category : Clothing	Base: R599	Final: R688.85	Stock: 120	Type: Taxable
ID: 2	Name: Wi-Fi Range Extender	Category : Electronics	Base: R1499	Final: R1723.85	Stock: 80	Type: Taxable

Figure 30 An Extract Showing the Stock levels for 1 and 2 BEFORE Order Processing

ID: 1	Name: Gloves Touchscreen Compatible	Category : Clothing	Base: R599	Final: R688.85	Stock: 0	Type: Taxable
ID: 2	Name: Wi-Fi Range Extender	Category : Electronics	Base: R1499	Final: R1723.85	Stock: 0	Type: Taxable

Figure 29 An Extract Showing the Stock Levels for Item 1 and 2 AFTER Order Processing

User Flow

The system follows a structured Cycle with Sub-menus with options(See Application for Full Details):

Startup Phase

Inventory loaded from CSV then the System initialises which then allows the User to Access the System.

Menu Interaction

User selects numeric options and the System executes the operation and returns to menu once completed

Order Processing Phase

User is Prompted to enter the number of Threads (Up to 32)

Concurrent Execution

Worker threads process orders in parallel. Inventory updated in real time. Results displayed with thread identifiers. Once Completed, a summary of the orders is displayed.

Analytics Phase

The User can generate Global and warehouse specific reports via the Analytics Sub-Menus. Export of these Analytics is can also be done through this.

Shutdown Phase

InventorySystem state saved and terminates safely

```
||---- Inventory Management System ----||
Powered by Group Twelve Software Solutions LTD

||---- Main Menu (Enter 1 - 9) ----||

1. Add a new product
2. Remove a product
3. Edit a product
4. Display all products
5. Search
6. Sort products
7. Process orders (Multi-Threaded)
8. Analytics
9. Save & Shutdown

Enter option: |
```

Figure 26 The Systems Main Menu

Multi-Threaded Execution Evidence

During processing, the interface exposes concurrency through explicit thread identifiers, showing both warehouse (producer) and worker (consumer) activity. This can be seen in the screenshot below

[Worker 21	Thread 27]	Order #21	Warehouse 1	Items: 4	SUCCESS
[Worker 10	Thread 16]	Order #10	Warehouse 1	Items: 4	SUCCESS
[Worker 12	Thread 18]	Order #31	Warehouse 1	Items: 1	SUCCESS
[Worker 24	Thread 30]	Order #16	Warehouse 1	Items: 3	SUCCESS

Figure 27 A log detailing the Worker ID, Thread ID, Order Details and Order Status

It should be noted that while, we do not show the stock levels within the Order Log, however, this can be checked before and after Order processing via Option 4: Display.

System Functionality & Testing

System Overview

Nexus Logistics is a multi-threaded inventory management system developed in C++. The system manages a product catalogue of 999 items across 11 categories, processes orders from 5 warehouses concurrently, and provides detailed analytics on system performance. It operates through a command-line interface (CLI) with 9 menu options, while also exposing internal concurrency behaviour through thread-level logging.

On startup, the system loads product data from a CSV file and immediately initialises 5 warehouse threads to load order data into a shared order pool. Once this phase is complete, the system transitions to user interaction through the main menu. The design ensures a clear separation between initialization, user interaction, and concurrent processing phases.

Feature Breakdown

Product Management (Options 1–3)

The system supports full CRUD operations on the inventory. Users can add, remove, and edit products by selecting options from the main menu.

When adding a product, the user selects a product type (Standard, Taxable, or Discounted), assigns a name, chooses from 11 categories, and defines price and quantity. A strict business rule enforces that Standard (zero-rated) products may only belong to the Perishables category. Any violation results in immediate rejection and no object creation.

Removal can be performed either by product ID or by searching via name. In both cases, the system displays product details before confirmation to prevent accidental deletion.

Editing is handled via a sub-menu allowing modification of name, category, price, or quantity. The system enforces category constraints, preventing invalid reassignment (e.g., non-Perishables to Perishables). All changes are immediately reflected in the inventory display.

Display and Search (Options 4–5)

Option 4 displays the full inventory. To ensure thread safety, the system copies product data into a temporary snapshot while holding a mutex lock, then releases the lock before printing. This prevents long console output from blocking concurrent operations.

Option 5 provides search functionality by name or ID. Name-based search uses a linear scan with partial matching using `std::string::find`, while ID-based search uses direct map lookup. Search results display product ID, name, category, price, and stock level.

Sorting (Option 6)

Sorting allows products to be ordered by price, quantity, type, name, or category. The system copies inventory data into a vector and applies `std::sort` with a custom comparator. The original map remains unchanged and continues to serve as the primary data structure keyed by product ID.

Multi-Threaded Order Processing (Option 7)

This is the core concurrent feature of the system. During Phase 1, five warehouse threads load orders from CSV files into a shared `OrderPool`. A total of 1,300 order entries are processed into the queue.

During Phase 2, the user selects the number of worker threads (1–32). Worker threads then consume orders concurrently from the shared queue. Each worker processes cart items by calling

InventorySystem::fulfillOrder(), which locks the inventory mutex and performs atomic stock deduction.

Each order is classified as:

-  **SUCCESS:** all items fulfilled
-  **PARTIAL:** some items fulfilled
-  **FAILED:** no items fulfilled

A 1ms delay simulates real-world processing latency. A final summary report displays thread count, total processed orders, success/failure breakdown, and execution time in milliseconds.

[Analytics \(Option 8\)](#)

The AnalyticsManager records detailed AnalyticsRecord entries during processing. Each record stores order ID, product ID, warehouse ID, quantity, category, product type, final price, and outcome.

Analytics provides:

-  Global reports (all warehouses combined)
-  Warehouse-specific reports (filtered by warehouse ID)
-  Export functionality to CSV using filesystem-based directory creation

Reports include revenue, lost revenue, category performance, product-type breakdown, failure rates, and most popular items.

[Save and Shutdown \(Option 9\)](#)

The system exports inventory to Catalog/Updated_Products.csv. During export, dynamic_cast is used to determine derived product types (Taxable, Discounted) to correctly write type-specific attributes such as tax or discount rates.

The original CSV file is preserved to maintain test consistency.

[Input Validation](#)

All user input is validated using helper functions readInt() and readDouble(). These functions handle invalid input by clearing the input stream state and flushing invalid buffer data using cin.clear() and cin.ignore().

This prevents invalid input (e.g. alphabetic characters in numeric fields) from causing persistent input failure states. Menu options also include boundary validation to prevent invalid selections.

[Testing Approach](#)

Testing was conducted manually through structured CLI interaction. Each feature was validated through normal and edge-case scenarios. The system was tested continuously during development and verified in final build form.

Test Scenario	Input / Action	Expected Result	Outcome
Add Standard product to Electronics	Type=Standard, Category=Electronics	System rejects with error message	Pass
Add Taxable product	Type=Taxable, valid inputs	Product added with correct ID	Pass
Remove product with confirmation	Select Remove, enter ID, confirm Y	Product removed from map	Pass
Remove product, cancel	Select Remove, enter ID, confirm N	Product remains in inventory	Pass
Search by partial name	Enter partial keyword	Matching products returned	Pass
Search with no results	Enter non-existent keyword	No products found message shown	Pass
Sort by price	Select sort by price	Products displayed in ascending price order	Pass
Process orders with 4 threads	Enter 4 for thread count	All orders processed, summary printed	Pass
Process orders with 16 threads	Enter 16 for thread count	Faster processing, same results	Pass
Process on empty pool	Run process again after first run	System shows pool is empty message	Pass
Analytics before processing	Select Option 8 without processing	System shows error, requires processing first	Pass
Global analytics report	Process orders then run global report	Revenue, categories, and stats displayed	Pass
Export analytics to CSV	Select export from analytics menu	CSV files created in Analytics folder	Pass
Invalid menu input (letters)	Type 'abc' at menu prompt	Error message, menu re-displayed	Pass
Save and shutdown	Select Option 9	CSV exported, program exits	Pass
Edit product category restrictions	Try changing Perishable category	System blocks the change	Pass
Concurrent stock deduction	Multiple threads fulfilling same product	No negative stock, no race conditions	Pass

Figure 31 A Table Outlining Tests Conducted on the System

Concurrent Order Testing

Concurrency was tested by executing order processing with varying thread counts (4, 8, 16, 32). Results confirmed:

- ✚ No race conditions in inventory updates
- ✚ No negative stock values
- ✚ Consistent order classification across runs
- ✚ Correct aggregation in analytics output

Multiple worker threads frequently accessed the same products simultaneously. The InventorySystem mutex ensured atomic stock updates, preventing inconsistent reads or double deductions.

Inventory Consistency Validation

Inventory consistency was validated by comparing:

- ✚ Initial stock values from CSV
- ✚ Final stock after processing
- ✚ Analytics-reported quantities sold

All values matched expected results, confirming that:

- ✚ No stock was lost due to race conditions
- ✚ No duplicate deductions occurred
- ✚ All fulfilled orders were correctly recorded

Even under maximum thread load (32 workers), inventory state remained stable and deterministic.

Summary

The system was thoroughly tested under normal operation, edge cases, and concurrent execution scenarios. Results confirm that all features function correctly, including product management, order processing, analytics generation, and file export.

Concurrency testing verifies that the system safely handles multiple threads accessing shared resources without corruption or inconsistency. Inventory integrity remains preserved under all tested conditions, demonstrating correct implementation of thread synchronization and resource management.

Performance & Limitations

Performance Observations

The system was timed during order processing using `std::chrono::high_resolution_clock`. Processing 1,300 orders with a 1ms simulated delay per order produced the following approximate results across different thread counts.

Thread Count	Approximate Time (ms)	Speedup vs 1 Thread
1	~20 000	1.0x (baseline)
4	~5 000	~4.0x
8	~2 600	~7.7x
16	~1 300	~15.4x
32	~650	~30.8x

The speedup scales well up to around 32 threads, after which the gains diminish because of hardware limitations on testing machine. The 1ms sleep per order is the

dominant cost, so adding threads effectively distributes this fixed-time work. Without the simulated delay, the actual computation is so fast that thread scaling is not visible in the timing.

Scalability Considerations

The system scales effectively due to a producer–consumer design where warehouse threads populate a shared order pool and worker threads process orders independently.

Scalability is limited by:

-  CPU core availability
-  Contention on inventory mutex
-  Condition variable wake-up overhead

Performance increases until thread count approaches physical core count, after which diminishing returns occur due to scheduling overhead rather than computation limits. The design is suitable for medium-scale workloads but could be extended using finer-grained locking or partitioned inventory systems.

Limitations and Assumptions

Search functionality uses a linear scan over product names since the system is indexed by product ID. This is efficient for ~1,000 products but does not scale well to larger datasets. A secondary indexing structure would improve lookup performance. The system relies on CSV-based storage with no database integration. While sufficient for simulation, this limits scalability, concurrency handling, and data integrity in production environments.

Analytics are computed on demand by iterating over stored records, introducing overhead during report generation. Pre-aggregation would improve performance for larger datasets. The system assumes a single-machine environment with shared memory concurrency. It does not support distributed processing. Thread count is capped at 32 to match typical consumer hardware constraints and to avoid excessive context switching.

Conclusion

The system demonstrates strong concurrent performance with predictable scaling under multi-threaded workloads. It efficiently utilises CPU resources while maintaining correctness through synchronisation mechanisms. However, scalability is limited by shared resource contention and file-based storage assumptions.

Challenges & Solutions

Building a multi-threaded C++ system introduced non-deterministic behaviour that made debugging and design validation significantly more complex than single-threaded development. The main challenges encountered were related to race conditions, STL misuse under concurrency, debugging unpredictable thread behaviour, and architectural changes required during implementation.

Debugging Multi-Threaded Code

Debugging concurrent code was difficult due to non-deterministic execution. A key issue occurred when duplicate order IDs appeared only under full warehouse thread execution. This was a classic race condition where two threads read and incremented the same static counter simultaneously, producing identical IDs.

The bug was initially difficult to reproduce because adding logging changed thread timing, causing the issue to disappear (Heisenbug behaviour). The solution was to instrument thread-aware logging using `std::this_thread::get_id()` and worker IDs, which revealed the exact interleaving pattern.

The final fix replaced the shared integer counter with:

- ✚ `std::atomic<int> nextID`
- ✚ `fetch_add(1)` to ensure atomic, lock-free increments

This eliminated the read-modify-write race entirely.

Race Conditions and Deadlock Prevention

Three major race conditions were identified and resolved:

Race Condition	Root Cause	Resolution
Stock decrement race	Two workers both see stock = 1, both pass the availability check, and both decrement — producing stock = -1	<code>inventoryMutex</code> held across the entire <code>fulfillOrder()</code> body, making the read-check-decrement sequence atomic
Order ID collision	Two warehouse threads read the same <code>nextID</code> value before either has incremented it	<code>std::atomic<int></code> with <code>fetch_add(1)</code> — lock-free on all modern architectures
Queue pointer corruption	Concurrent push/pop on a raw <code>std::queue</code> corrupts internal linked-list pointers	All queue operations routed through <code>poolMutex</code> ; consumers block on a <code>condition_variable</code> rather than spinning

Deadlocks were prevented through a strict design rule: each function acquires at most one mutex at a time. Combined with RAII (`std::lock_guard`), this guarantees no cyclic-wait conditions and ensures automatic release even under exceptions.

STL Misuse and Refactoring

Early development incorrectly assumed STL containers were thread-safe. This caused several issues:

- ✚ OrderPool: Initial design used a raw queue with polling, causing CPU waste and race windows. Refactored to producer-consumer model using `condition_variable`, eliminating busy-waiting.
- ✚ InventorySystem: Unprotected `std::map` access caused undefined behaviour under concurrent reads/writes. Fixed by introducing `inventoryMutex` and ensuring all public access is locked.

- ✚ AnalyticsManager: std::vector reallocations risked invalidating references during concurrent writes. Resolved using lock_guard around all push_back operations.

These issues highlighted that STL safety guarantees do not extend to concurrent access without explicit synchronisation.

Design Changes During Implementation

The most significant architectural change was a shift from a warehouse-centric design to a strict producer-consumer model.

Initial Design

Each warehouse thread:

- ✚ Loaded orders
- ✚ Processed orders
- ✚ Updated inventory directly
- ✚ Wrote separate output logs

This caused high contention and unclear ownership of shared resources.

Final Design

The system was restructured as:

- ✚ Warehouse threads (producers): Only parse CSV files and push Order objects into a shared OrderPool
- ✚ Worker threads (consumers): Process orders and update inventory through controlled access
- ✚ Central InventorySystem: Single shared authority for stock updates
- ✚ AnalyticsManager: Unified reporting system replacing per-warehouse logs

This separation significantly reduced synchronisation complexity and improved scalability.

- ✚ Additional mid-implementation changes included:
- ✚ Introduction of a StandardProduct class due to business rules restricting zero-rated items to Perishables
- ✚ Addition of product categories across CLI, sorting, analytics, and CSV export logic
- ✚ Expansion of dynamic_cast usage for correct type-specific export behaviour

These changes confirmed the importance of designing shared data structures and business rules early, as late modifications propagated across multiple system layers

Summary

The primary challenges in this system stemmed from concurrency complexity, undefined STL behaviour under multi-threaded access, and evolving architectural requirements. These were resolved through atomic operations, mutex-based synchronisation, and a redesigned producer-consumer architecture. The final system demonstrates stable, deterministic behaviour under concurrent load while maintaining correctness and scalability.

Individual Contribution Summary

Member Name	Student ID	Role	Contribution Description	% Contribution
Yahya Bobat	223020664	Lead Developer	Involved with the implementation of the System from the Designed Classed Provided and the Multi-Threading Implementation	15
Jawad Kachbal	223037045	High-Level System Design Flow & Developer	Involved with the Foundational High-Level Design of the System as well, Implementation of the Analytics functions	15
Yusuf Sheikh	224068106	Documentation, Presentation	Responsible for Project Documentation, Presentation Preparation	14
Mahomed Valli Mahomed	224005878	Overall Class & System Design and Diagram Design	Involved with the Foundational Class Design from the High-Level System Flow, Diagrams	14
Muhammad Varachia	224004509	Overall Class & System Design and Presentation	Involved with the Foundational Class Design from the High-Level System Flow and Assisted with Presentations	14
Kiyan Krishna	224115592	Overall Assistant in various Categories and System Testing	Flexible Role that assisted all Group Members in need. Responsible for System Testing	14
Andiswa Hlengwa	221002430	GUI Design and Implementation	Responsible for the Design and Implementation of the GUI	14

Conclusion

Final System Overview

The Multi-Threaded Inventory and Order Processing System — Nexus Logistics — is a fully functional C++ application that manages a catalogue of 1,000 products across 11 categories, loads and processes 1,300 orders from five warehouses concurrently, and delivers detailed analytics on sales performance, revenue, and order outcomes. The system operates through a command-line interface with nine menu options and supports between 1 and 32 configurable worker threads. All core objectives defined in the project proposal have been met, and the final build reflects a level of technical rigour that goes beyond the initial scope in several meaningful areas.

The system is structured around a producer-consumer model that cleanly separates its two phases of execution. Five warehouse threads load orders into a shared OrderPool during Phase 1; a user-selected number of worker threads drain and fulfil those orders during Phase 2. The two phases are temporally separated so that warehouse and worker threads never compete at runtime, and the pool's loadingComplete flag is safely reset via prepareForProcessing() to support repeat processing runs. The product hierarchy is built on an abstract Product base class with TaxableProduct, DiscountedProduct, and StandardProduct as concrete subclasses, each enforcing type-specific pricing logic and category business rules through overridden virtual methods.

An AnalyticsManager collects per-item records during processing and supports both global and warehouse-scoped reporting across metrics including revenue, lost revenue, category performance, product-type breakdowns, failure rates, and order status summaries, with full CSV export capability. All shared state is protected through purpose-appropriate synchronisation primitives: a mutex paired with a condition variable on the order queue, a coarse-grained mutex on the inventory map, std::atomic for lock-free order ID generation, and a dedicated print mutex that serialises console output without impeding core processing throughput

Technical Achievements

The most significant technical achievement is a verified, deadlock-free concurrency design that holds under all tested thread counts. Final stock totals match expected values exactly across every run, confirming that no updates are lost and no shared state is silently corrupted, the strongest practical evidence of correctness that a concurrent system can offer short of formal verification. Performance scales near-linearly with thread count: wall-clock processing time drops from approximately 20,000 ms on a single thread to roughly 650 ms on 32 threads, a 30x speedup that demonstrates genuine parallelism rather than the false concurrency that excessive locking or busy-waiting would produce. Memory safety is maintained throughout via std::unique_ptr for exclusive ownership of worker threads and heap objects, controlled snapshot copying to keep critical sections short and lock contention low, and RAII lock guards that release mutexes unconditionally, even when exceptions are thrown.

The analytics subsystem represents a substantial scope expansion over the original proposal: by capturing per-item records rather than per-order summaries during processing, it computes fine-grained metrics by product category, product type, and warehouse that would have been impossible from coarser data. The addition of the StandardProduct subclass and the product category attribute, though both introduced mid-implementation, are now fully integrated across the CLI, the CSV loader, the export logic, and the analytics pipeline without visible seams

Lessons Learned

Multi-threaded development demands a fundamentally different approach to both design and debugging than single-threaded coursework prepares a team for. The most important lesson was that concurrency does not compose transparently: two pieces of code that are each correct in isolation can produce undefined behaviour when interleaved, and discovering this requires reasoning about invariants and happens-before relationships rather than tracing execution line by line. Thread-aware logging, emitting the OS thread ID alongside every completed order, proved more effective than any traditional debugger for diagnosing non-deterministic faults. The team learned to treat it as a first-class debugging instrument rather than an afterthought. The project also reinforced how costly late design changes become when a data attribute, such as the product category field, cuts across every layer of the system simultaneously. Identifying cross-cutting concerns early and expressing them explicitly in the class model before concrete implementations solidify would have substantially reduced the refactoring burden the team absorbed mid-sprint.

STL containers carry no thread-safety guarantees whatsoever and treating them as though they do is a silent error that may only surface under specific scheduling conditions on specific hardware, a lesson that stuck precisely because the team experienced it rather than read about it. Finally, RAII idioms such as `lock_guard`, `unique_ptr`, are not stylistic conventions but engineering tools that eliminate whole categories of latent faults by making correctness the path of least resistance. A mutex that cannot be forgotten to unlock, and a heap allocation that cannot be forgotten to free, are defects that simply cannot exist.

AI Usage

Artificial Intelligence tools, specifically ChatGPT, were used during the development of this project as a supplementary aid to support understanding, refinement, and documentation of concepts. AI was not used to automatically generate complete solutions or replace core development work, but rather to assist in improving clarity, structure, and technical accuracy.

AI assistance was primarily used in the following areas:

- ✚ Refining written explanations for report sections such as concurrency design, memory management, and system architecture
- ✚ Improving formatting and structure to meet academic and submission standards
- ✚ Clarifying theoretical concepts related to multi-threading, synchronisation, and C++ best practices
- ✚ Reviewing and suggesting improvements to existing code (provided by the team) without directly generating full implementations

All code included in the final system was written, tested, and validated by the team. Any AI-generated suggestions were critically evaluated, adapted where necessary, and integrated only after ensuring correctness and alignment with the system design.

AI was not used during formal testing or evaluation of system correctness. All test cases, debugging processes, and performance validation were conducted independently by the team.

This approach ensured that AI functioned strictly as a support tool, while all design decisions, implementation, and problem-solving remained the responsibility of the project team.

References

- cppreference.com. (2024). *std::atomic*. Retrieved from CPP Reference:
<https://en.cppreference.com/w/cpp/atomic/atomic>
- cppreference.com. (2024). *std::chrono::high_resolution_clock*. Retrieved from CPP Reference:
https://en.cppreference.com/w/cpp/chrono/high_resolution_clock
- cppreference.com. (2024). *std::condition_variable*. Retrieved from CPP Reference:
https://en.cppreference.com/w/cpp/thread/condition_variable
- cppreference.com. (2024). *std::mutex*. Retrieved from cppreference.com:
<https://en.cppreference.com/w/cpp/thread/mutex>
- cppreference.com. (2024). *std::shared_ptr*. Retrieved from CPP Reference:
https://en.cppreference.com/w/cpp/memory/shared_ptr
- cppreference.com. (2024). *std::thread*. Retrieved from CPP Reference:
<https://en.cppreference.com/w/cpp/thread/thread>
- cppreference.com. (2024). *std::unique_ptr*. Retrieved from CPP Reference:
https://en.cppreference.com/w/cpp/memory/unique_ptr
- cppreference.com. (n.d.). *std::lock_guard*. Retrieved from 2024:
https://en.cppreference.com/w/cpp/thread/lock_guard
- ISO/IEC. (2020). ISO/IEC 14882:2020 — Programming Languages — C++. In ISO/IEC. International Organization for Standardization.
- Meyers, S. (2014). Items 35–42 on concurrency. In S. Meyers, *Effective Modern C++*. O'Reilly Media.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). In Addison-Wesley, *The C++ Programming Language (4th ed.)*.
- Williams, A. (2019). In A. Williams, *C++ Concurrency in Action: Practical Multithreading (2nd ed.)*. Manning Publications.